第1讲 编译导论

1.1 编译器基础定义

编译器：将**高级语言(C/C++/Python/Rust/Java)翻译成机器语言(ISA)**，供CPU/GPU/NPU/DSP/FPGA硬件执行。

- 高级语言：贴近人类思维，开发效率高；
- 机器语言：硬件直接执行，编程繁琐易错。

1.2 C程序完整编译运行流程

C源程序 →【预处理器】`.i` →【C编译器】`.s`汇编 →【汇编器】`.o`目标文件 →【链接器Linker】(链接库代码) → 机器可执行代码

1. 预处理器：处理`#include`头文件、`#define`宏定义；
2. 编译器：C源码翻译生成汇编代码；
3. 汇编器：汇编语言翻译成机器目标文件`.o`；
4. 链接器：合并多个目标文件+系统库，解析外部符号，生成可执行程序。

1.3 GCC / Clang 优化等级

优化等级	说明
`-O0`	默认，**无任何优化**，只减少编译时间
`-O1 / -O`	少量优化，效果介于-O0与-O2之间
`-O2`	开启大多数中级优化，平衡编译开销与代码性能
`-Os`	基于-O2，优化目标为缩减生成代码体积
`-O3`	大量高级优化；以编译时间、内存、代码大小换取最高性能
`-Ofast`	-O3基础上开启激进优化，**不严格遵守语言标准**

> PPT示例：十亿次循环累加，-O2对比-O0性能可提升数十万倍。

1.4 编译器七大阶段（前端 + 后端）

共享组件：**符号表 Symbol Table**、错误处理模块 Error Handler

- 符号表：记录`<token_name, attribute_value>`，保存变量、函数、类型信息，编译器全阶段访问。

前端（源语言相关、机器无关）

1. **词法分析**：字符流 → Token记号流
    - clang查看token命令：`clang -cc1 -dump-tokens xx.c`
2. **语法分析**：Token流 → 语法树(AST)
    - clang查看AST命令：`clang -fsyntax-only -Xclang -ast-dump xx.c`
3. **语义分析**：遍历语法树；做**类型检查、隐式类型转换**，输出带注解语法树。
4. **中间代码生成**：生成机器无关IR（三地址码 / LLVM-IR）

- 生成LLVM IR命令：`clang -cc1 xx.c -emit-llvm -o xx.ll`

后端（硬件相关）

5. **机器无关代码优化**：在IR层面做优化，不依赖目标硬件。
   - 示例：常量折叠、公共子表达式消除、循环不变量外提
   - clang优化IR命令：`clang -S -emit-llvm -O3 test.c -o test-new.ll`
6. **目标代码生成**：IR翻译汇编代码；完成指令选择、寄存器分配
   - llc工具命令：`llc-14 xxx.ll -o xxx.s`
7. **机器相关代码优化**：针对特定CPU架构优化汇编指令。

> 现代编译器分层：
>
> - 前端：词法、语法、语义、中间代码生成
> - 后端：IR优化、目标代码生成

1.5 AI编译栈（PPT拓展）

应用(CV/NLP/大模型) → 语言前端(Python/模型文件) → 图编译器GraphIR → 算子编译器TensorIR → Codegen(LLVM IR) → CPU/GPU/NPU/DSP硬件

1.6 课程实验大纲（了解）

Lab0环境搭建｜Lab1词法语法解析器(Flex+Bison)｜Lab2 IR生成｜Lab3 LA64龙芯后端｜Lab4 SSA转换｜Lab5机器无关优化(常量传播、不变式外提)｜Lab6寄存器分配

词法分析（重点）

2.1 基本概念

1. **词法分析器(扫描器 Scanner)**：输入源程序字符流，输出Token记号流。
2. Token记号格式：`<记号名，属性值>`。
3. 词法单元：关键字、标识符、常量、运算符、分隔符。
4. 词法错误：非法字符；**不处理语法错误、语义错误**。

2.2 正则表达式 Regular Expression(RE)

正则表达式用来描述词法单元模式。 运算符优先级（高→低）：`*`闭包 > 连接 > `|`或

- `r*`：0次或多次重复；`r+`1次及以上；`r?`0或1次；
- `rs`：r和s连接；`r|s` r或者s。

2.3 有限自动机 FA

1. **NFA 非确定有限自动机**
   - 允许ε空转移；同一个状态+同一个输入字符，可转移到**多个后继状态**。
2. **DFA 确定有限自动机**
   - **无ε转移**；任意状态+输入字符，**仅有唯一后继状态**。词法分析程序实际执行使用DFA。

2.4 正则到DFA完整转换流程

正则表达式RE → Thompson构造算法生成NFA → 子集构造算法得到DFA → DFA最小化（合并等价状态）。

2.5 Flex工具

输入：正则表达式规则；输出：C语言实现的词法分析器。

> 考点区分： 词法：正则、NFA/DFA；语法：上下文无关文法CFG。

语法分析总述（重点）

两大类语法分析技术：

1. **自顶向下分析**：从开始符号出发，做**最左推导**；代表：递归下降、LL(1)预测分析。
2. **自底向上分析**：对输入串**归约**，归约是**最右推导的逆过程**；代表：移进-归约、SLR(1)、LR(1)。

3.1 上下文无关文法 CFG

文法四元组 $G = (V_T, V_N, S, P)$

- V_T：终结符集合（Token）
- V_N：非终结符集合（语法变量）
- S：文法开始符号，$S \in V_N$
- P：产生式集合 $A \to \alpha$；$A \in V_N$，$\alpha \in (V_T \cup V_N)^*$

推导

1. **最左推导**：每一步替换句型**最左边**的非终结符；对应自顶向下分析。
2. **最右推导（规范推导）**：每一步替换句型**最右边**的非终结符；自底向上分析，归约就是最右推导的逆过程。
3. **二义文法**：同一个句子存在**两棵不同语法分析树**。

FIRST集合

$FIRST(\alpha)$：串α可以推导得到的**首终结符集合**；若$\alpha \Rightarrow^* \varepsilon$，则$\varepsilon \in FIRST(\alpha)$。

FIRST计算规则

1. 若$\alpha = a\beta$，a是终结符：$a \in FIRST(\alpha)$。
2. 若$\alpha = A\beta$，A是非终结符：
   - 将 $FIRST(A) - \{\varepsilon\}$ 全部加入 $FIRST(\alpha)$；
   - 如果 $\varepsilon \in FIRST(A)$，把 $FIRST(\beta)$ 全部加入 $FIRST(\alpha)$。

FOLLOW集合

$FOLLOW(A)$：某些句型中紧跟在非终结符A后面的终结符集合；若A出现在句型末尾，则结束符`$ \in FOLLOW(A)`。

FOLLOW计算规则

1. 开始符号S：`$ \in FOLLOW(S)`。
2. 若存在产生式 $B \to \alpha A \beta$：

- $FIRST(\beta)-\{\varepsilon\}$ 加入 $FOLLOW(A)$；
- 如果 $\varepsilon \in FIRST(\beta)$，把 $FOLLOW(B)$全部加入$FOLLOW(A)$。

3. 若存在产生式 $B \to \alpha A$：$FOLLOW(B)$全部加入$FOLLOW(A)$。

SELECT集合（LL(1)预测集合）

对产生式 $A \to \alpha$

$$SELECT(A \to \alpha) = \begin{cases} FIRST(\alpha) & \varepsilon \notin FIRST(\alpha) \\ (FIRST(\alpha)-\{\varepsilon\}) \cup FOLLOW(A) & \varepsilon \in FIRST(\alpha) \end{cases}$$

LL(1)文法判定条件

同一个非终结符A，任意两条不同产生式 $A \to \alpha$，$A \to \beta$：

$$SELECT(A \to \alpha) \cap SELECT(A \to \beta) = \emptyset$$

交集为空，则文法是LL(1)文法。

> LL(1)含义：L从左向右扫描输入；L执行最左推导；1代表向前看1个输入符号。

LL(1)文法预处理

1. **消除左递归**：分为直接左递归、间接左递归；
2. **提取左公因子**，消除公共前缀。

LL(1)非递归预测分析器

组成：输入缓冲区、分析栈、**预测分析表M**

- 栈初始化：压入结束符$，再压入开始符号S；
- 预测表$M[A,a]$：行=非终结符，列=终结符/$；单元格存放产生式；空白代表语法错误。

自底向上语法分析

4.1 归约

归约操作：句型中子串β，依据产生式$A \to \beta$替换为A。

> 核心定理：**归约是最右推导的逆过程**。

4.2 句柄 Handle

句型 $\gamma\beta w$，w全部为终结符；若 $S \Rightarrow_{rm}^{*} \gamma A w \Rightarrow_{rm} \gamma\beta w$，子串$\beta$叫做该句型的句柄。 性质：

1. 句柄的**右边只能是终结符**；
2. **二义文法句柄不唯一**。

4.3 移进-归约分析（栈实现）

数据结构：栈保存已经扫描的文法符号；输入缓冲区保存尚未处理的Token。 四种动作：

1. **移进 shift**：将当前输入符号压入栈；输入指针向后移动。
2. **归约 reduce**：识别栈顶句柄；弹出句柄符号，压入产生式左部非终结符；**输入指针保持不动**。
3. **接受 accept**：输入全部处理完毕，分析成功。
4. **报错 error**：检测语法错误。

两类冲突（考试高频）

1. **移进-归约冲突**：当前格局既可以移进输入符号，又可以对栈顶执行归约；典型：`if-then-else`二义文法。
2. **归约-归约冲突**：栈顶存在多个候选句柄，多条产生式均可执行归约。

> 普通移进-归约无法解决冲突，引出LR系列分析方法。

4.4 LR分析总览

LR含义：**L：从左向右扫描输入；R：构造最右推导的逆（归约）** 文法能力包含关系：$LR(0) \subset SLR(1) \subset LR(1)$

LR分析器通用结构

1. 栈：成对存储「状态s_i，文法符号X_i」，栈底为初始状态s_0。
2. 两张分析表
   - $action[状态, 终结符]$：`sj`移进状态j；`rj`按第j号产生式归约；`acc`接受；空白=error
   - $goto[状态, 非终结符]$：归约完成，压入非终结符之后跳转的目标状态。

LR分析算法步骤

1. 栈初始化：压入初始状态s_0；输入指针指向第一个token。
2. 循环查询 $action[栈顶状态，\ 当前输入符号]$
   1. `sj`移进：输入符号、状态j压栈；输入指针+1。
   2. `rj`归约：产生式$A \to \beta$，$k = |\beta|$；从栈弹出k对（符号、状态）；取弹出后的栈顶状态s_t；查询$goto[s_t, A] = s_j$；将A与s_j压栈；**输入指针不移动**。
   3. `acc`：分析成功，结束。
   4. error：语法错误处理。

4.5 活前缀（可行前缀 viable prefix）

> 活前缀：某个最右句型的前缀，并且**不超过句柄的右端**。 重要性质：LR分析栈中出现的符号串一定是活前缀；LR的DFA功能就是识别文法全部活前缀。

4.6 LR(0)项目

LR(0)项目：在产生式右部插入标记点$\cdot$，标记分析到什么位置。 示例：产生式 $A \to XYZ$，得到4个LR(0)项目：

$$A \to \cdot XYZ, \quad A \to X \cdot YZ, \quad A \to XY \cdot Z, \quad A \to XYZ \cdot$$

- 点$\cdot$左边：已经识别完成的符号；点右边：期待后续输入的符号。
- ε空产生式仅有1个项目：$A \to \cdot$。

4.7 拓广文法 Augmented Grammar

新增拓广开始符号S'，增加产生式：$S' \to S$。 目的：只有归约到项目 $S' \to S\cdot$，才判定输入串完全合法；避免原开始符号多次出现造成误判。

4.8 两大基础函数（SLR、LR(1)全部使用）

① closure(I) 闭包运算

输入项目集合I，输出闭包集合：

1. 将I中所有项目加入闭包；
2. 如果项目 $A \to \alpha \cdot B\beta$ 在闭包中，且$B \to \gamma$是文法产生式；把项目 $B \to \cdot\gamma$加入集合。

② goto(I,X) 状态转移函数

I为项目集，X为文法符号（终结符/非终结符）

1. J初始为空集合；
2. 取I中全部项目 $A \to \alpha \cdot X\beta$，改写为 $A \to \alpha X \cdot \beta$，放入J；
3. 返回 `closure(J)`。

> 项目集规范族：反复迭代`closure`、`goto`，得到DFA全部状态集合。

4.9 SLR(1) Simple-LR分析表构造

1. 对G构造拓广文法G'。
2. 计算LR(0)项目集规范族，得到识别活前缀的DFA。
3. 构造`action`表：
   - 若 $[A \to \alpha \cdot a\beta] \in I_i$，$goto(I_i, a) = I_j$，设置 $action[i, a] = s_j$；
   - 若 $[A \to \alpha\cdot] \in I_i$，$\forall a \in FOLLOW(A)$，设置 $action[i, a] = r_j$；
   - 若 $[S' \to S\cdot] \in I_i$，设置 $action[i, \$] = acc$。
4. 构造`goto`表：对非终结符A，若$goto(I_i, A) = I_j$，设置$goto[i, A] = j$。
5. 表格未定义位置标记error。

> SLR(1)文法：构造SLR分析表，**没有移进-归约冲突、归约-归约冲突**。 SLR缺陷：归约仅依靠FOLLOW集合，忽略上下文；部分无二义文法会产生虚假冲突。

4.10 LR(1) 规范LR

> 改进SLR缺陷：项目增加**向前看搜索符**。 LR(1)项目格式：$[A \to \alpha \cdot \beta,\ a]$

- 前半部分与LR(0)项目完全一致；
- a为向前看搜索符号（终结符或`$`）；
- 仅当点在产生式最右端$\beta = \varepsilon$时搜索符生效：**只有当前输入符号等于a，才允许执行本产生式归约**。

> LR(1)的数字1代表向前看1个符号。

LR(1) closure(I)闭包

1. I内全部项目加入闭包；

2. 如果 $[A \to \alpha \cdot B\beta, a] \in closure(I)$，$B \to \gamma$； 对每一个 $b \in FIRST(\beta a)$，将项目$[B \to \cdot\gamma, b]$加入闭包。

LR(1) goto(I,X)

1. J置空；
2. I中项目 $[A \to \alpha \cdot X\beta, a]$ →改写 $[A \to \alpha X \cdot \beta, a]$放入J；
3. 返回`closure(J)`。

LR(1)分析表构造

1. 拓广文法，构造LR(1)项目集规范族。
2. action表：
   - $[A \to \alpha \cdot a\beta, b] \in I_i$，$goto(I_i, a) = I_j$，$action[i, a] = s_j$；
   - $[A \to \alpha \cdot, a] \in I_i$ 且 $A \neq S'$，设置 $action[i, a] = r_j$；**此处不再使用FOLLOW集合**；
   - $[S' \to S \cdot, \$] \in I_i$，设置$action[i, \$] = acc$。
3. goto表规则与SLR完全相同。
4. 空白表格项填写error。

> LR(1)优缺点：表达能力强，可以处理SLR无法处理的无二义文法；**缺点：项目集状态数量爆炸，手工计算繁琐**。

LL(1) vs SLR/LR(1)对比表

对比维度	LL(1) 自顶向下	LR(SLR/LR(1))自底向上
推导	最左推导	最右推导（归约是逆过程）
识别条件	看到产生式右部第一个终结符	识别完整句柄（整个产生式右部）
文法约束	必须消除左递归、提取左公因子	几乎全部上下文无关文法
栈存储	仅文法符号	状态 + 文法符号（状态携带上下文）
核心集合/函数	`FIRST、FOLLOW、SELECT`	`closure、goto`；SLR用FOLLOW；LR(1)使用向前看搜索符
分析表	非终结符 × 终结符	状态 × 终结符；LR(1)状态数量爆炸

语义分析（简要）

1. 核心任务：类型检查、隐式转换、符号表查找、作用域处理。
2. 属性文法：语法树结点附加属性
   - **综合属性**：由子结点属性计算得到，自底向上计算；
   - **继承属性**：由父/兄弟结点传入，自顶向下传递。

中间代码、代码生成（简要）

1. 三地址码（四元式）：`t = op1 op op2`，大量临时变量t；
2. SSA静态单赋值：每个变量只赋值一次，方便优化；LLVM-IR采用SSA；

3. 机器无关优化：常量折叠、常量传播、公共子表达式消除CSE、循环不变量外提、死代码删除；
4. 目标代码生成：指令选择、寄存器分配；寄存器不足时变量溢出spill到内存栈。